

编程基础 II — 可计算性

软件工程与计算 I · 2026 · 南京大学

| 刘钦

Reference

- 《逻辑的引擎》 Martin Davis
- 《Turing, His Machine and Computability》 Jürg Kohlas
- 《Register Machines are Turing Machines》 Colin B.Price
- 《A Relatively Small Turing Machine Whose Behavior Is Independent of Set Theory》 Adam Yedidia Scott Aaronson

知识点

- 停机问题的证明
- 归约：从已知问题证明新问题不可判定
- 图灵机的理解与定义
- 通用图灵机的定义
- 图灵机的等价模型
- 可计算函数 vs 实际可计算函数
- 可识别 vs 可判定（半可判定性）
- Rice 定理与程序分析的极限
- 加法的递归定义
- 图灵机的数字表示
- RegisterMachine 的最小组成

Outline

可计算性

- 逻辑发展简史
- 罗素悖论
- 停机问题
- 可识别 vs 可判定
- 归约与 Rice 定理
- 哥德尔不完备性定理
- 邱奇-图灵论题
- 不可计算

可计算模型

- μ -递归函数
- 图灵机
- Register Machine

课后练习

- 练习 1–5: 停机问题、归约、Rice 定理、图灵完备
- **练习 6: 用代码模拟图灵机**
(Python 实现 + 补充 rules)

核心问题

为什么计算机**可以**计算？

哪些是计算机**可以**计算的？

可不可以用**数学方法**来证明？

可计算性

逻辑发展简史 · 罗素悖论 · 停机问题 · 哥德尔不完备性定理

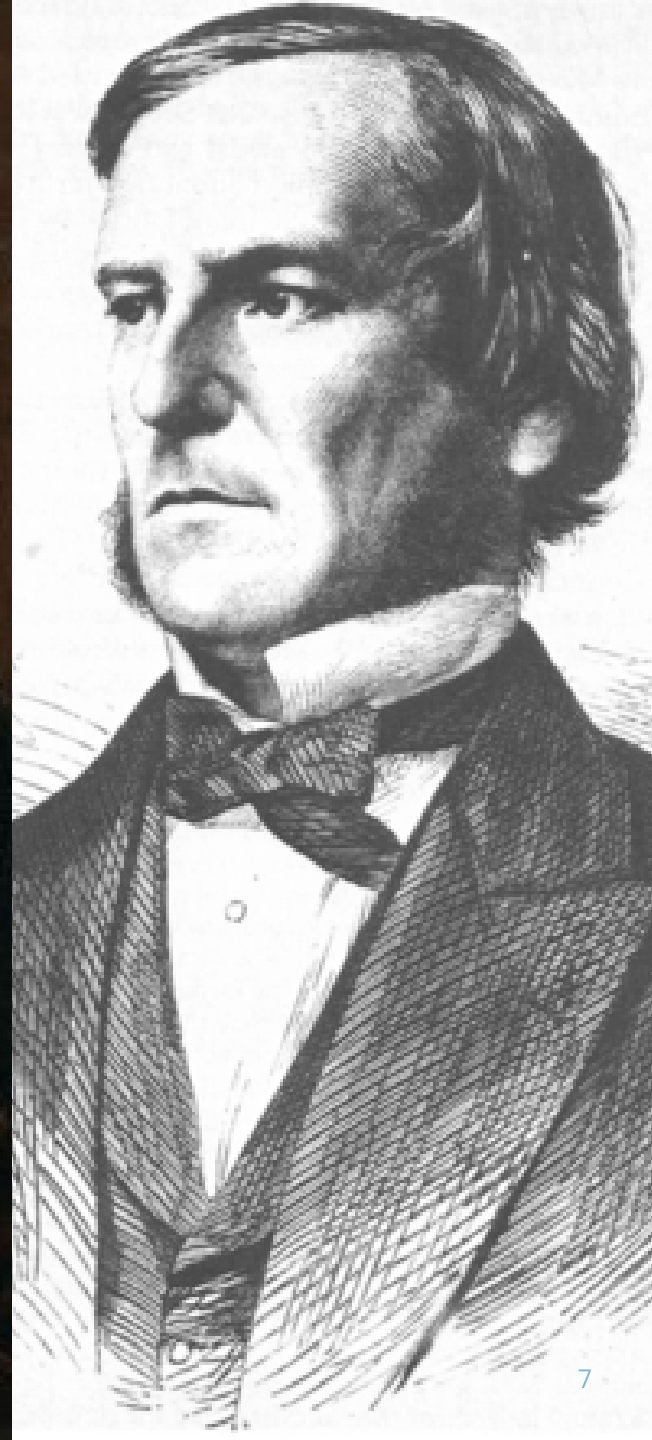
莱布尼兹之梦 & 布尔代数

莱布尼兹

1. 创造涵盖人类知识全部范围的**百科全书**
2. 对关键概念提供合适的**符号**
3. **推理演算** (calculus rationcinator)

布尔

- 布尔代数
- 证明了**逻辑演绎**可以成为**数学**的一个分支
- 离完备性的目标还有距离



数理逻辑

弗雷格

- 形式化的句法
 - 全称量词 (All)、存在量词 (exist)
 - 如果, 那么 / 非 / 或 / 且

第一次有一个精确的数理逻辑系统, 至少在原则上包含了数学家们通常使用的全部推理。

希望能够为自然数提出一种**纯粹逻辑的理论**, 从而证明一切数学是逻辑的一个分支。



无穷集和对角线法

康托尔

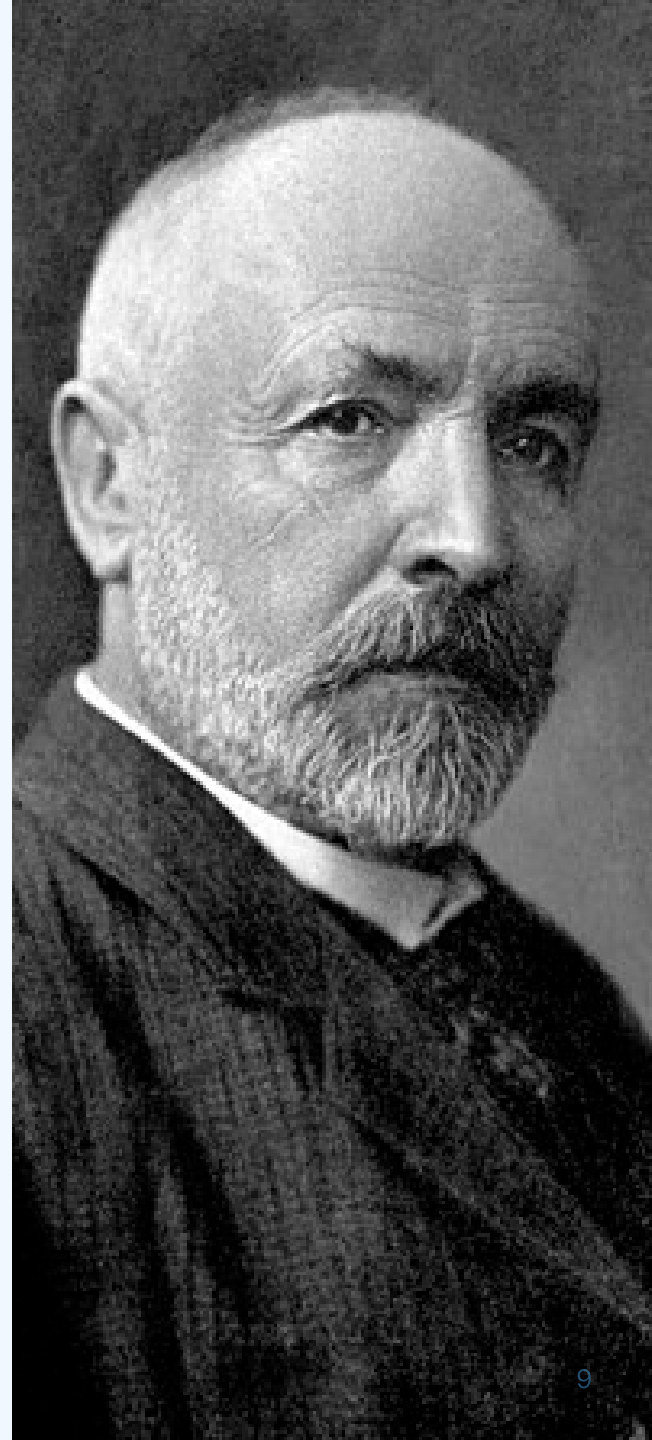
- 无穷集的大小是**不同的**
- 只要一个集合中的元素可以同另一个集合中的元素**一一对应**，我们就可以说这个集合的元素数目相等

对角线法

(包裹上的标签，就是包裹的东西)

(可以找出一个新包裹和任何贴标签的包裹都不一样)

- 一切自然数集所组成的集合的**基数** 要大于 **自然数的基数**



希尔伯特判定问题

判定问题：试图找到清楚明白的验算程序，只要用所谓的一阶逻辑的符号系统写出来的某些前提和所提出的结果给定，那么通过这些程序就是可以判定的。

希尔伯特是乐观的，他的墓碑上刻着两行字：

我们必须知道
我们将会知道



理发师悖论

在某个城市中有一位理发师，他的广告词是这样写的：「本人将为本城所有不给自己刮脸的人刮脸，我也只给这些人刮脸。」

有一天，这位理发师看见自己的胡子长了——他能不能给自己刮脸呢？

如果他不给自己刮脸 →
他属于「不给自己刮脸的人」 →
他要给自己刮脸

如果他给自己刮脸 →
他属于「给自己刮脸的人」 →
他不该给自己刮脸

矛盾！

罗素悖论

设命题函数 $P(x)$ 表示 " $x \in x$ ", 假设由性质 P 确定了一个类 $A = \{x \mid x \in x\}$

$A \in A$ 是否成立?

若 $A \in A$

- A 是 A 的元素
- A 不具有性质 P
- $A \notin A$ 矛盾!

若 $A \notin A$

- A 具有性质 P
- A 是由所有具有性质 P 的类组成的
- $A \in A$ 矛盾!



停机问题

不存在这样一个程序（算法），它能够计算任何程序（算法）在给定输入上是否会结束（停机）。

哥德尔使得计划落空

- 和爱因斯坦是挚友，常一起散步（1906—1978）
- 1930 年博士论文中证明了弗雷格的规则是**完备的**

哥德尔的不完备定理

「我不可在系统 T 内被证明」

- 在皮亚诺算术（PA）中，存在命题**既不能被证明为真，也不能被否定**——希尔伯特的「任何命题皆可判定」目标**无法实现**

冯诺依曼意识到希尔伯特纲领是错的。



停机问题解决了希尔伯特判定问题

- 对角线方法将允许我们构造出一个与图灵机的任何停机集合都不同的自然数集合，称它为 D
- 集合 D 不是任何图灵机的停机集合

证明思路

- 判定问题的**肯定**解答将会为解决**所有数学问题**提供一种算法
- 但蕴含着：只要有一个数学问题可以被证明在算法上是不可解的，那么判定问题本身就**必定不可解**
- 「找到一种算法，判定一个给定的自然数是否属于集合 D 」—— 这就是一个不可解问题的例子

为什么集合 D 的判定问题不可解？

「找到一种算法，判定一个给定的自然数是否属于集合 D 。」

反证法：假设存在这样的算法，则存在这样的图灵机——

- 把输入的数属于 D ，那么按以前运转
- 否则将永远右移（2 个五元组即可表达）

这台新机器的停机集合就是 D 。

然而， D 被构造为「与任何图灵机的停机集合都不同的集合」——

矛盾！ 所以这样的算法不可能存在。

证明停机问题 · 1

假设存在一个万能算法 **God_algo**: 给它一段程序和输入, 它就能告诉你这段程序是否会停机。

```
bool God_algo(char* program, char* input)
{
    if (<program> halts on <input>)
        return true;
    return false;
}
```

假设 `if` 的判断语句里面是你天才思考的结晶, 它能够像上帝一样洞察一切程序的宿命。

证明停机问题 · 2

从 God_algo 出发，导出一个新的算法：

```
bool Satan_algo(char* program)
{
    if (God_algo(program, program)) {
        while (1);          // loop forever!
        return false;       // can never get here!
    }
    else
        return true;
}
```

证明停机问题 · 3

当我们把这个算法运用到它**自身**身上时，会发生什么？

```
Satan_algo(Satan_algo);
```

Satan_algo(Satan_algo) 能停机

→ God_algo 返回 true

→ 进入死循环，**不能停机**

矛盾！

Satan_algo(Satan_algo) 不能停机

→ God_algo 返回 false

→ 返回 true，**能停机**

矛盾！

所以 **God_algo 不存在**。正如拉格朗日所说：「陛下，我们不需要（上帝）这个假设」。

补充阅读：Lambda 演算的证明

Halting 是一个假设存在的停机求解器。考察表达式：

```
E = Halting( $\lambda m.$ not(Halting(m,m)),  $\lambda m.$ not(Halting(m,m)))
```

- 若 E 返回 **True**，则对函数自身应用，得到 `not(Halting(...)) = False`，矛盾！
- 若 E 返回 **False**，则同理推出 **True**，矛盾！

This contradiction shows that the halting problem solver **Halting** cannot exist.

可识别 vs 可判定 (Semi-decidability)

停机问题的"不对称性"

如果程序**确实停机**，运行它，有限步内就能知道 
如果程序**不停机**，我们只能永远等下去 

	停机	不停机
可判定 (decidable)	能确认	能确认
可识别 (recognizable)	能确认	无法确认

停机问题：可识别，但不可判定

为什么这个区别重要？

```
// 你的静态分析工具发现了死循环：  
while (condition()) { ... }  
  
// 它能做到的最好情况：  
//  如果确实有死循环 → 可以报告"发现疑似死循环"  
//  如果没有死循环 → 无法保证"一定没有死循环"
```

这就是为什么所有静态分析工具 (FindBugs、SonarQube)
都不能保证零漏报——理论上限制，不是工程缺陷

归约（Reduction） — 证明不可判定性的工具

核心思想

若问题 A 可归约到 B,
则"能解 B"意味着"能解 A";
逆否命题: A 不可解 $\rightarrow$ B 也不可解

停机问题（已知不可判定）
↓ 归约
新问题（证明也不可判定）

经典归约链

问题	不可判定原因
程序是否有死循环	← 停机问题归约
两段程序是否等价	← 停机问题归约
程序是否会抛异常	← 停机问题归约

归约的直觉

给任意程序 Q 和输入 I, 构造:

```
def P():  
    Q(I)                # 只做一件事: 运行 Q(I)  
    return "done"      # 若到达这里, 说明 Q(I) 停机了
```

Q(I) 的行为	P 的行为	死循环检测结论
Q(I) 停机	P 正常返回	P 无死循环
Q(I) 不停机	P 永远卡在 Q(I) 里	P 有死循环

- 死循环检测(P) 的答案 = Q(I) 是否不停机
- 能检测死循环 = 能判断停机问题
- 矛盾! 死循环检测不可判定

归约推理 — 逐步拆解

目标：证明「死循环检测」不可判定

假设（反证）：存在一个神奇工具 `LOOP(P)`，能判断任意程序是否有死循环。

第一步：拿到任意停机问题实例 (Q, I)

问题： $Q(I)$ 到底会不会停机？

第二步：构造辅助程序 P

```
def P():  
    Q(I)  
    return "done"
```

第三步：调用 `LOOP(P)`

`LOOP(P)` = NO（无死循环） $\rightarrow$ $Q(I)$ 停机 ✓

`LOOP(P)` = YES（有死循环） $\rightarrow$ $Q(I)$ 不停机 ✓

第四步：`LOOP(P)` 的输出直接回答了停机问题！

但停机问题不可判定 $\rightarrow$ `LOOP` 不可能存在 $\rightarrow$ 矛盾 ✗

关键： P 的死循环行为与 $Q(I)$ 是否停机一一对应，
所以判断 P 的死循环 = 判断 $Q(I)$ 停机，难度相同。

归约的本质 — 为什么这样推理有效?

归约 = 问题转化

停机问题

↕ 难度等价

死循环检测问题

若 A 可归约到 B ($A \leq B$):

- 解 B 的工具 $\rightarrow$ 可以解 A
- A 不可解 $\rightarrow$ B 也不可解 (逆否)

常见误解

✗ 误: P 里加个 `while True:` 就有死循环了

```
def P():  
    Q(I)  
    while True: pass    # ← 这里有问题
```

问题: 若 Q(I) 不停机, P 永远到不了 `while True` ——两种情况都"死循环", 无法区分。

✓ 正确: 不加 `while True`, 让 P 的死循环完全来自 Q(I), 才能建立精确对应。

Rice 定理 — 软件工程师必知的不可判定性结论

定理内容

Rice 定理： 程序的任何非平凡语义属性都是不可判定的。

"语义属性"：与程序做什么有关（输出、行为）
"非平凡"：不是所有程序都有，也不是所有程序都没有

- 以下全部不可判定：
- ✗ 程序是否会停机
 - ✗ 程序是否会输出 "Hello"
 - ✗ 程序是否包含 bug
 - ✗ 程序对所有输入是否与另一程序等价
 - ✗ 程序是否会访问空指针

能判定什么？

语法属性（与代码结构有关）是可判定的：

- ✓ 程序有多少行代码
- ✓ 程序声明了多少个变量
- ✓ 程序是否符合语法规则
- ✓ 程序的圈复杂度

对软件工程的启示

工具	能做到	做不到
编译器类型检查	✓ 语法语义类型	✗ 运行时所有 bug
静态分析器	✓ 启发式检测	✗ 零误报零漏报
测试	✓ 验证已测用例	✗ 证明所有输入正确

Rice 定理是"程序分析存在本质局限"的理论根基

哥德尔不完备性定理

第一不完备定理：任何相容的形式系统，只要蕴涵皮亚诺算术公理，就可以在其中构造在体系中既不能证明也不能否证的命题（即体系是**不完备的**）。

第二不完备定理：任何相容的形式系统，只要蕴涵皮亚诺算术公理，它就**不能**用于证明它本身的**相容性**。

哥德尔不完备性定理的证明 · 1

哥德尔构造了一个命题，用自然语言表达就是：

「P 不可在系统 T 内证明」（即「我不可以被证明」）

若 T 可以证明 P

→ 因为 P 说的是"P 不可证"

→ T 无法证明 P 矛盾！

∴ T 不可以证明 P

P 说的正是 "T 不可证明 P"

∴ P 是**正确的命题**，同时**无法**由 T 内证明！

这番证明位于 T 系统之外，用到了"T 是一致（无矛盾）的"这个前提——并非 T 系统里面的内容。

哥德尔不完备性定理的证明 · 2

哥德尔构造了公式 $G(n): \text{UnPr}(N(n))$ ("N(n) 在 T 中不可证")

1. 因为 G 也是一个 well-formed formula, 它有自身的编码 g (一个自然数)
2. 把 g 代入 G 的自由变量 n , 得到命题:

$$G(g): \text{UnPr}(G(g))$$

用自然语言: 「我是不可在 T 内证明的」

这样, 我们就在形式系统 T 内, 表达出了哥德尔的核心命题, 从而证明了不完备性定理。

图灵构想通用计算机

受哥德尔不完备定理启发

图灵的通用计算机

- 五元组（初始状态，注视符号，改写符号，移动方向，变化状态）
- 停机问题解决了**希尔伯特判定问题**
- 一台图灵机单凭自身就可以完成**任何图灵机**可能做到的任何事情
- 图灵提出了**机器、程序、数据**观念的融合与替换
- 图灵很快证明了他的可计算性概念与邱奇的 **λ 可定义性是等价的**



通用图灵机

- 任何图灵机都能表示为一串字符串 M
- I 是输入数据

通用图灵机 U 能够在任意输入 I 上模拟任意图灵机 M

如果 $M \langle I \rangle$ 接受
→ $U \langle M, I \rangle$ 接受

如果 $M \langle I \rangle$ 拒绝
→ $U \langle M, I \rangle$ 拒绝

邱奇-图灵论题

The Church—Turing thesis states that a function is **algorithmically computable** if and only if it is **computable by a Turing machine**.

- According to the Church—Turing thesis, computable functions are exactly the functions that **can be calculated using a mechanical calculation device** given unlimited amounts of time and storage space.
- Equivalently, this thesis states that any function **which has an algorithm** is computable.
- Note that an algorithm is understood to be **a sequence of steps a person** with unlimited time and an infinite supply of pen and paper could follow.



邱奇-图灵论题：三条独立路径

| λ 演算

Alonzo Church

美国数学家

定义函数的 λ 演算方法

| 图灵机

Alan Turing

英国数学家

可以从输入执行计算的理论机器模型

| 递归函数

Kurt Gödel

奥地利-美国数学家

值可以通过递归计算的函数类形式定义

三种计算过程（递归、 λ 演算、图灵机）被证明是等价的

等价的计算模型

可计算函数的类可以用多种等价的计算模型来定义：

- **图灵机** (Turing machines)
- **μ -递归函数** (μ -recursive functions)
- **Lambda 演算** (Lambda calculus)
- **Post 机** (Post-Turing machines and tag machines)
- **寄存器机** (Register machines)

有没有注意到重要的假设

... given **unlimited** amounts of **time** and **storage space**

可计算函数 $\neq$ 实际可计算函数

有没有理论上不可计算的函数？

忙碌的海狸（Busy Beaver）

busy beaver 图灵机的性质：

1. 符号数（色数）为 2：图灵机有两大属性——符号数和状态数，色数最小值为 2
2. 初始输入全为空白，「接受状态」仅有「停机状态」
3. 在所有能停机的 2 色- n 状态图灵机中，运行步数最多的图灵机

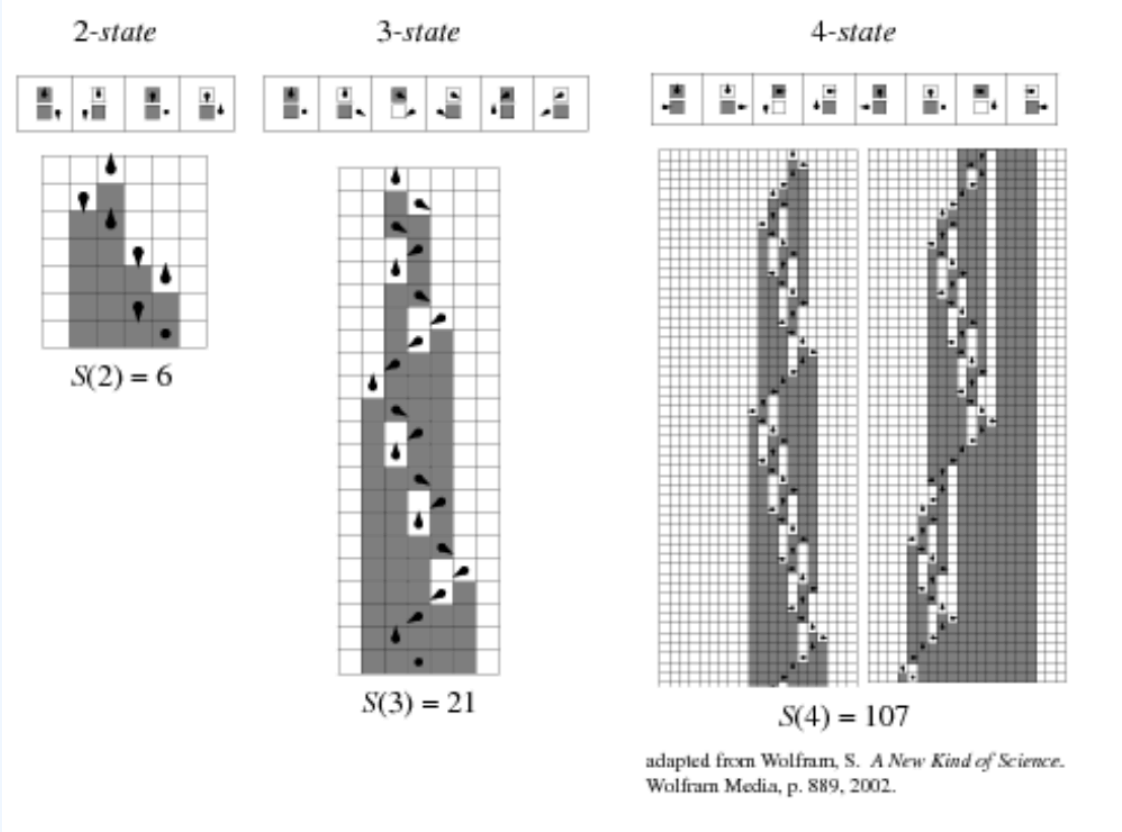
BB(n)：2 色- n 状态的可停机图灵机的最大运行步数

BB(n) 是一个快速增长序列

目前只算出来 4 个值：

n	BB(n)
1	1
2	6
3	21
4	107

BB(n) 增长速度超过任何可计算函数——这就是为什么它本身是不可计算的



哥德巴赫猜想的证明

Adam Yedidia 把哥德巴赫猜想编码为有 **4888 个状态** 的 Busy Beaver 图灵机：

- 逻辑：从小到大验证每一个偶数是否能表示为两个质数之和
- 如果发现不能表示为两个质数之和的偶数，则**停机**

```
START start_read_:  
  a -> start_write_next; R; b
```

如果让这台图灵机跑起来，只要跑了 BB(4888) 步后还没有停机，那就**证明了哥德巴赫猜想为真！**

他还写了一个验证**黎曼猜想**的图灵机，有 **5372 个状态**。

可计算模型

μ -递归函数 · 图灵机 · Register Machine

μ -递归函数

什么是自然数？

自然数的皮亚诺公理系统

1. 1 是自然数
2. 每一个确定的自然数 a ，都有一个确定的后继数 a' ， a' 也是自然数
3. 如果自然数 b 、 c 的后继数都是 a ，那么 $b = c$
4. 1 不是任何自然数的后继数
5. 任意关于自然数的命题，如果证明了它对 1 是对的，又假定它对 n 为真时，可以证明它对 n' 也真，那么命题对所有自然数都真。(数学归纳法)

若将 0 也视作自然数，则公理中的 1 要换成 0。



函数是一种映射关系

从输入值集合 X 到可能的输出值集合 Y 的函数 $f: X \rightarrow Y$ 是 X 与 Y 的关系，满足：

1. **完全的**：对集合 X 中任一元素 x 都有集合 Y 中的元素 y 满足 xfy
2. **多对一的**：若 $f(x) = y$ 且 $f(x) = z$ ，则 $y = z$

我们能表达什么样的映射？

- 恒等映射： $f(x) = x$
- 后继映射： $f(x) = S(x)$ // $S(x)$ 表示 x 的后继
- 递归定义： $f(1) = 1$, $f(x) = S(f(P(x)))$ // $P(x)$ 表示 x 的前继

原始递归函数

基本函数：

函数	定义
常数函数	0 元常数函数 0 是原始递归的
后继函数 S	接受一个参数，返回后继数
投影函数 P_i^n	接受 n 个参数，返回第 i 个

运算：

运算	描述
复合	$h(\bar{x}) = f(g_1(\bar{x}), \dots, g_k(\bar{x}))$
原始递归	$h(0, \bar{x}) = f(\bar{x}) ; \quad h(S(n), \bar{x}) = g(h(n, \bar{x}), n, \bar{x})$

加法的递归定义

直觉上我们把加法递归定义为：

$$\begin{aligned}\text{add}(0, x) &= x \\ \text{add}(n+1, x) &= \text{add}(n, x) + 1\end{aligned}$$

严格的原始递归定义：

$$\begin{aligned}\text{add}(0, x) &= P_1^1(x) \\ \text{add}(S(n), x) &= S(P_1^3(\text{add}(n, x), n, x))\end{aligned}$$

注意： P_1^3 是一个接受 3 个参数并返回第一个的投影函数。

减法的递归定义

首先定义前驱函数：

$$\begin{aligned}\text{pred}(0) &= 0 \\ \text{pred}(S(n)) &= P_2^2(\text{pred}(n), n)\end{aligned}$$

然后定义有限减法（截止到 0）：

$$\begin{aligned}\text{sub}(0, x) &= P_1^1(x) \\ \text{sub}(S(n), x) &= \text{pred}(P_1^3(\text{sub}(n, x), n, x))\end{aligned}$$

阿克曼函数 (1928)

Wilhelm Ackermann (David Hilbert 的学生) 发现 x 的 y 次幂的 z -重积分 $A(x,y,z)$ 是递归的但不是原始递归的。

$$A(m,n) = \begin{cases} n + 1 & \text{if } m = 0 \\ A(m - 1, 1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{if } m > 0 \text{ and } n > 0 \end{cases}$$

m\n	0	1	2	3	n
0	1	2	3	4	n+1
1	2	3	4	5	n+2
2	3	5	7	9	2·(n+3)−3
3	5	13	29	61	2^(n+3)−3
4	13	65533	2^65536−3	...	...



μ-递归函数

在原始递归函数的基础上，增加 μ算子：

编号	函数/算子	描述
(1)	常数函数	$f(x_1, \dots, x_k) = n$
(2)	后继函数 S	$S(x) = x + 1$
(3)	投影函数 P_i^k	$P_i^k(x_1, \dots, x_k) = x_i$
(4)	复合算子	$h(\bar{x}) = f(g_1(\bar{x}), \dots, g_m(\bar{x}))$
(5)	原始递归算子	$h(0, \bar{x}) = g(\bar{x}) ; \quad h(y+1, \bar{x}) = h(y, f(y, \dots), \bar{x})$
(6)	μ算子	返回使 $f(y, \bar{x}) = 0$ 的最小 y

为什么需要 μ 算子?

原始递归函数都是全函数 (total function) —— 对任意输入都有定义, 且必然停机。

但阿克曼函数证明: 存在可计算却不是原始递归的函数。

原始递归的局限

递归展开的层数在定义时就已固定:

$$\begin{aligned}h(0, x) &= f(x) \\ h(n+1, x) &= g(h(n, x), n, x)\end{aligned}$$

递归深度 = n , 有界, 必然停机

μ 算子做了什么

$$\mu y [f(y, x) = 0]$$

搜索使 $f(y, x) = 0$ 的最小 y :

- $y=0$ 试试 $\rightarrow$ 不对
- $y=1$ 试试 $\rightarrow$ 不对
- $y=2$ 试试 $\rightarrow \dots$

若永远找不到, 则无定义 (不停机)

关键: μ 算子引入了无界搜索, 搜索深度在运行前未知, 因此函数可能对某些输入没有定义——成为部分函数 (partial function)。

可计算函数 = 部分可计算函数

「可计算函数」在形式理论中，默认指**部分可计算函数**——

全函数 vs 部分函数

类型	含义
全函数 (total)	对所有输入都有定义，必然停机
部分函数 (partial)	仅对部分输入有定义，其余不停机

原始递归函数 $\subseteq$ **全可计算函数** $\subseteq$ **部分可计算函数**

为什么要允许不停机？

- 图灵机对某些输入可能**永远运行**
- 停机问题告诉我们：**无法**用算法判断一台图灵机是否会停机
- 若强制要求所有可计算函数都是全函数，则该函数类**无法被形式化封闭**

允许部分函数，才能让 μ -递归 = λ 演算 = 图灵机**三者等价**成立。

直觉上「可计算」意味着「能算出答案」，但形式定义中「可计算」意味着「存在一个算法去尝试计算」——它可能永远算不完。

μ算子 与 阿克曼函数的关系

阿克曼函数 $A(m,n)$ 增长速度**超过任何原始递归函数**:

- 原始递归函数能计算 $A(m,n)$ 的每一个值 (停机)
- 但无法用**有界**的递归层次来定义整个 A ——需要**嵌套递归**, 深度随 m 增长而爆炸

原始递归 $\subset$ μ -递归

类别	特点
原始递归函数	全函数, 递归有界
μ -递归函数	可以是部分函数, 递归无界

邱奇-图灵论题的意义

μ -递归函数 = 图灵机可计算函数

"可计算" 的精确数学定义就是 μ -递归函数, 而不是原始递归函数

阿克曼函数是**全函数** (每个值都有定义), 但它揭示了: **全函数 $\neq$ 原始递归**, 需要 μ -递归框架才能完整描述所有可计算函数。

图灵机

图灵机

图灵机由以下部分组成：

- 一个无限长的纸带（磁带）
- 一个读写头
- 一个状态寄存器
- 一个有限的指令集（转移函数表）

当前状态 s	输入 i	输出动作 o	下一状态 s'
B	1	前移	C
A	0	写 1	B
C	0	后移	A

图灵机的定义

一台图灵机是一个七元有序组 $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ ，其中：

元素	说明
Q	非空有穷状态集合
Σ	非空有穷输入字母表，不含空白符 $\square$
Γ	非空有穷带字母表， $\Sigma \subset \Gamma$ ， $\square \in \Gamma - \Sigma$
$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, \text{—}\}$	转移函数
$q_0 \in Q$	起始状态
$q_{accept} \in Q$	接受状态
$q_{reject} \in Q$	拒绝状态， $q_{reject} \neq q_{accept}$

七元组 vs 五元组

七元组（现代标准定义）

$$(Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$$

- 显式区分接受与拒绝两种停机状态
- 转移函数: $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, -\}$
- 适合描述语言识别（接受/拒绝一个字符串）
- 教材主流，用于可计算性/复杂度理论

五元组（图灵原始形式）

$$(q, s_1, s_2, d, q')$$

字段	含义
q	当前状态
s ₁	读到的符号
s ₂	写入的符号
d	移动方向（L/R/N）
q'	新状态

- 每条指令即一个五元组，贴近硬件执行
- 停机 = 没有匹配的指令（无定义则停）
- 适合描述计算过程（如程序例子）

本质相同：五元组是七元组转移函数 δ 的逐条展开写法；七元组是五元组集合的结构化封装。

图灵机运行模式

- 开始时输入符号串 $\omega = \omega_0\omega_1...\omega_{n-1}$ 填在纸带的 $0,1,...,n-1$ 号格子上, 其余为 $\square$
- 读写头指向第 0 号格子, 机器处于状态 q_0

转移规则: 当状态为 q , 读写头读到符号 x , 设 $\delta(q, x) = (q', x', L)$, 则:

1. 机器进入新状态 q'
2. 将当前格子的符号改为 x'
3. 读写头**向左移动**一格

- 进入 **q_{accept}** $\rightarrow$ 立刻停机并**接受**
- 进入 **q_{reject}** $\rightarrow$ 立刻停机并**拒绝**
- 若 $\delta(q,x)$ 未定义 $\rightarrow$ 立刻**停机**

图灵机例子：3 + 2

设 $M = (\{0,1,10,11,E\}, \{0,1\}, \{0,1,\square\}, \delta, 0, E, \text{—})$ (E 为接受/停机状态)

0,0 -> 0,0R

1,0 -> 10,1R

10,0 -> 11,0L

11,0 -> E

0,1 -> 1,1R

1,1 -> 1,1R

10,1 -> 10,1R

11,1 -> 0,0S

磁带数据 0000001110110000 (表示 3+2)

步数	状态	磁带
1—7	0	0000001110110000
8	1	0000001110110000
11	10	0000001111110000
14	11	0000001111110000
15	0	0000001111100000 (停机)

图灵机例子： $f(x) = x + 1$

q1,0,1,l,q2; q1,1,0,l,q3; q1,b,b,n,q4;
q2,0,0,l,q2; q2,1,1,l,q2; q2,b,b,n,q4;
q3,0,1,l,q2; q3,1,0,l,q3; q3,b,b,n,q4.

五元组 (q1, s1, s2, r, q2) 含义：

元素	含义
q1	当前状态
s1	读写头读入的符号
s2	写入当前方格的符号
r/l/n	向右/向左/保持不动
q2	新状态

读写头一开始位于数据最右边一位，b 表示空格，q1 为初始状态，q4 为结束状态。

图灵机例子： $f(x) = 2^x$

当前状态	B	0	1
q1	1,L,q7	0,R,q1	1,R,q2
q2	B,R,q3	0,R,q2	1,R,q2
q3	0,L,q4	0,R,q3	Error
q4	B,L,q5	0,L,q4	Error
q5	Error	1,L,q5	0,L,q6
q6	B,R,q1	0,L,q6	1,L,q6
q7	Halt	B,L,q7	Error

实现思想：在二进制表示下，在原数后面添上一个 0 就是原来的数乘以二。

思考题：如何用数字表达一个图灵机？

A description of a single state in Z

1704 (1705bR|2476aR)

字段	含义
1704	状态名称
1705bR	读到 "a" 时：跳转到 1705 状态，写 b，右移
2476aR	读到 "b" 时：跳转到 2476 状态，写 a，右移

Register Machine

寄存器机与图灵机的等价性

图灵机 对应 寄存器机

图灵机

- 一条**无限纸带**
- 读写头指向当前格
- 当前符号：s
- 左侧内容：m（编码为整数）
- 右侧内容：n（编码为整数）

寄存器机

- 多个**寄存器** (m, s, n, z)
- 加上 ALU（算术逻辑单元）
- 寄存器 m、s、n 对应纸带状态
- 寄存器 z 始终为 0

图灵机的数字表示：读写头移动

读写头右移 ($s = 0$):

- $m' = 2 \times m$
- $s' = \text{remainder}(n / 2)$
- $n' = n / 2$ (整除)

读写头左移 ($s = 1$):

- $s' = \text{remainder}(n / 2)$
- $m' = 2 \times m + s$
- $n' = n / 2$ (整除)

$$s^* = \text{rem}(n/2) \quad m^* = s + 2m \quad n^* = \lfloor n/2 \rfloor$$

最小指令集

指令	含义
mov reg, 0	将寄存器 "reg" 置为 0
inc reg	寄存器 "reg" 加 1
decjmpreg reg, lab	若 "reg" 为 0 则跳转到 "lab", 否则减 1
jmp lab	无条件跳转到 "lab"
hlt	停机

实现 $m^* = s + 2m$ (m 乘 2)

```
    mov ecx, 3
    mov ebx, 0
L3: decjmpreg ecx, L4
    inc ebx
    inc ebx
    jmp L3
L4: hlt
```

实现 $s^* = \text{rem}(n/2)$, $n^* = n/2$

```
    mov ecx, 5
    mov eax, 0
L3:  mov ebx, 0
     decjmpreg ecx, L4
     inc ebx
     decjmpreg ecx, L4
     inc eax
     jmp L3
L4:  hlt
```

从 ecx=5 开始, 结束时 eax=2 (商), ebx=1 (余数)

读写头右移的完整代码

```
:   decjmpreg eax, L1           ; eax holds m
    inc edx
    inc edx                    ; z = 2m
    jmp L2

:   decjmpreg ebx, L3
    inc edx                    ; if s is 1, z = z + 1

:   decjmpreg edx, L4
    inc eax
    ; jmp L3                    ; m* = z

:   mov ebx, 0
    decjmpreg ecx, L6
    inc ebx
    decjmpreg ecx, L6
    inc edx
    ; jmp L4                    ; s* = rem(n/2), z = n/2

:   decjmpreg edx, L7
    inc ecx
    ; jmp L6                    ; n* = z

:   hlt
```

只用两条指令

将 `mov` 和 `jmp` 用 `decjmpreg` 替代：

- `mov reg, 0` → 用 `decjmpreg` 循环清零
- `jmp lab` → `decjmpreg esi, lab` (esi 始终为 0, 保证无条件跳转)

整个图灵机的运作，可以只用 `inc` 和 `decjmpreg` 两条指令实现！

只用两个寄存器

寄存器 r : 将三维坐标 (a, b, c) Gödelization 为单个数

$$r = GS(a, b, c) = 2^a \cdot 3^b \cdot 5^c$$

寄存器 z : 始终为 0

- `decjmpreg z, L` 实现无条件跳转

Theorem 1

对于每台图灵机 T ，存在一台寄存器机 R ，其行为与 T 完全相同。
这台寄存器机由两个寄存器和两条指令组成。

所有等价的计算模型，都在计算同一件事——

Effective Computability = Turing Computability

总结

本讲总结

可计算性

问题	结论
希尔伯特判定问题	$\times$ 不可解
停机问题	$\times$ 不可判定，但 可识别
归约 (Reduction)	从停机问题出发证明其他问题不可判定
Rice 定理	程序的任何非平凡语义属性不可判定
哥德尔不完备定理	形式系统必有真但不可证的命题
邱奇-图灵论题	可计算 = 图灵机可计算

可计算模型

模型	核心思想
μ -递归函数	无界搜索 $\rightarrow$ 部分函数
λ 演算	函数抽象与应用
图灵机	纸带 + 读写头 + 状态转移
Register Machine	寄存器 + 算术运算

四者等价：都精确刻画了同一个"可计算"概念

核心洞见回顾

停机问题：用对角线法构造矛盾——任何声称能判定停机的算法，都能被构造出一个让它自相矛盾的程序。

哥德尔定理：在任何足够强的形式系统内，都存在「我不可被证明」这样的命题——系统的一致性决定了它的不完备性。

邱奇-图灵论题： μ -递归、 λ 演算、图灵机三条独立路径，最终汇聚于同一个「可计算」的边界。

归约 (Reduction)：将停机问题"变形"嵌入新问题，证明"能解新问题 $\rightarrow$ 能解停机问题"——从而推出新问题也不可判定。

图灵机代码练习：状态转移表 = `rules` 字典；理解图灵机，就理解了"算法"最本质的形式定义。

Rice 定理：程序的任何非平凡语义属性都不可判定——静态分析工具、bug 检测器存在本质局限，不是工程问题，而是数学证明。

Register Machine：仅需 2 个寄存器 + 2 条指令即可完全模拟图灵机。

课后练习

思考 · 推导 · 体会"边界"

练习 1 — 停机问题的自毁

下面的伪代码能否正确运行？请分析会发生什么。

```
def will_halt(program, input):  
    # 假设它能完美判断 program(input) 是否停机  
    ...  
  
def evil(program):  
    if will_halt(program, program):  
        while True: pass    # 永远循环  
    else:  
        return              # 停机
```

问题： `evil(evil)` 会停机吗？

提示：分两种情况讨论，看看每种情况会推出什么结论。

练习 2 — 自指与悖论

罗素悖论、停机问题、哥德尔定理都利用了**自指（self-reference）**构造矛盾。

思考以下场景，哪些会产生悖论？ 哪些不会？

场景	是否悖论？ 原因？
一个程序打印自身的源代码（Quine）	
一个函数判断"自己的返回值是否为 True"	
一个集合包含"所有不包含自身的集合"	
一个网页链接到自己	

找出"自指产生悖论"和"自指不产生悖论"的关键区别。

练习 3 — 乘法的原始递归定义

已知加法的原始递归定义：

```
add(0, x)    = x
add(S(n), x) = S(add(n, x))
```

请类比加法，写出乘法 `mult(m, n)` 的原始递归定义。

提示：乘法的递归边界是什么？递归步骤中如何用加法表达"再加一行"？

```
mult(0, x)    = ?
mult(S(n), x) = ?
```

验证： `mult(S(S(0)), S(S(S(0))))` 是否等于 6？（即 2×3 ）

练习 4 — 可计算 or 不可计算？

判断下列问题是否可计算，并简要说明理由：

问题	可计算？	理由
给定整数 n，判断 n 是否为质数		
给定 Java 程序，判断它是否包含死循环		
给定 Java 程序，统计它有多少个变量声明		
给定整数 n，计算第 n 个斐波那契数		
给定两段程序，判断它们对所有输入是否输出相同结果		

哪些问题等价于停机问题？

练习 5 — 图灵完备

JavaScript、Python、Java 都是**图灵完备 (Turing-complete) **的语言。

结合邱奇-图灵论题，回答：

1. "图灵完备"到底意味着什么？一门语言需要具备哪些最低限度的能力？
2. 以下"语言"或系统是否图灵完备？为什么？
 - SQL (关系型数据库查询语言)
 - HTML + CSS (不含 JavaScript)
 - 正则表达式 (Regular Expressions)
 - Excel 公式
3. 如果一门语言**不是**图灵完备的，它能做什么？不能做什么？有什么好处？

练习 6 — 用代码模拟图灵机 (1/2)

图灵机的五要素：状态集、字母表、转移规则、初始状态、停机状态。

下面用 Python 字典实现转移规则表：

```
def turing_machine(tape, rules, initial_state, halt_states):
    """
    tape:          list of symbols, e.g. ['1','1','B','1','1']
    rules:          dict { (state, symbol): (new_symbol, direction, new_state) }
                    direction: 'R' (右移) or 'L' (左移)
    halt_states:    set of states where machine stops
    """
    head = 0
    state = initial_state
    while state not in halt_states:
        symbol = tape[head] if 0 <= head < len(tape) else 'B'
        if (state, symbol) not in rules:
            break
        new_sym, direction, new_state = rules[(state, symbol)]
        tape[head] = new_sym
        head += (1 if direction == 'R' else -1)
        state = new_state
    return tape, state
```

每一步：读当前符号 → 查 rules → 写新符号 + 移动读写头 + 切换状态

练习 6 — 用代码模拟图灵机 (2/2)

任务：补充 `rules` 字典，实现以下任意一个功能：

任务 A — 一元加法

输入磁带：1 1 B 1 1 1 (表示 $2 + 3$)

期望输出：1 1 1 1 1 B (5 个 1)

思路：把分隔符 B 替换为 1，再擦掉末尾一个 1

任务 B — 判断回文

输入磁带：1 0 1 0 1

期望：停机后首格写 Y (是回文) 或 N (否)

思路：反复比较首尾字符，标记已比较的位置

建议步骤：

1. 先在纸上画状态转移图 (圆圈=状态，箭头=规则)
2. 逐条翻译为 `rules` 字典的一行
3. 用 `turing_machine(tape, rules, 'q0', {'HALT'})` 验证

编程基础 II — 可计算性

There are more things in heaven and earth, Horatio,
than are dreamt of in your philosophy.

— Shakespeare, **Hamlet**